

BÁO CÁO ĐỒ ÁN CUỐI KỲ

TRIỂN KHAI HỆ THỐNG
MULTI-AGENT
TRÊN EDGE

Ứng dụng giám sát môi trường thông minh
với suy luận AI cục bộ

Học phần	Công nghệ điện đám mây và điện toán biên (CE2206)
Mã lớp	CE2206.CH201
Đề tài	Triển khai hệ thống Multi-Agent trên Edge
Miền ứng dụng	Giám sát môi trường thông minh
Sinh viên thực hiện	[Họ và tên]
Mã số sinh viên	[MSSV]
Giảng viên hướng dẫn	[Họ và tên giảng viên]
Cơ sở đào tạo	[Tên trường / Khoa]
Thời gian thực hiện	07/2026 – 08/2026
Ngày nộp	[dd/mm/yyyy]

Tóm tắt

Điện toán biên (*edge computing*) đặt suy luận và điều phối gần nguồn dữ liệu nhằm giảm độ trễ và phụ thuộc đường truyền lên đám mây. Đề án này thiết kế, triển khai và đánh giá định lượng một hệ **multi-agent** trên môi trường edge tài nguyên hạn chế (**2 vCPU / 4 GB RAM / CPU-only** cho mỗi agent), phục vụ miền **giám sát môi trường thông minh**.

Hệ thống gồm ba agent vai trò tách biệt: (i) **Sensor Agent** thu thập và chuẩn hóa dữ liệu cảm biến giả lập (nhiệt độ, độ ẩm, PM2.5, CO₂); (ii) **Analysis Agent** thực hiện **suy luận AI cục bộ** bằng Isolation Forest xuất **ONNX**, chạy trên **ONNX Runtime**; (iii) **Decision Agent** đóng vai **orchestrator tập trung nhẹ**, phát cảnh báo và xử lý lỗi khi Analysis timeout bằng fallback ngưỡng cứng (**degraded**). Toàn bộ giao tiếp đi qua **MQTT** (Eclipse Mosquitto) với schema JSON thống nhất và **trace_id** để đo độ trễ end-to-end (E2E). Báo cáo trình bày đầy đủ phân tích yêu cầu, thiết kế kiến trúc/giao thức/chính sách quyết định, hiện thực Docker đúng hạn mức đề bài, cùng thực nghiệm có giả thuyết và thảo luận trade-off.

Kết quả đo trên Docker Compose cho thấy: E2E latency **p50 $\approx$ 9,48 ms**, **p95 $\approx$ 49,72 ms**; thời gian inference ONNX **p50 $\approx$ 4,98 ms**; RAM đỉnh quan sát của Analysis Agent khoảng **51,5 MiB** — còn rất nhiều dư địa so với trần 4 GB. Khi Analysis bị ngắt, Decision vẫn phát alert **degraded** sau khoảng timeout cấu hình (5 s) mà không làm sập toàn hệ. Hệ thống kèm dashboard giám sát realtime và bộ script đo lường/fault-injection phục vụ demo và tái lập thí nghiệm.

Từ khóa: điện toán biên, multi-agent, MQTT, ONNX, phát hiện bất thường, giám sát môi trường, fault tolerance, đo lường hiệu năng.

Mục lục

Tóm tắt	i
Danh mục từ viết tắt	vi
1 Mở đầu	1
1.1 Đặt vấn đề	1
1.2 Mục tiêu đề án	2
1.3 Phạm vi nghiên cứu	2
1.4 Phương pháp thực hiện	3
1.5 Đóng góp chính	3
1.6 Cấu trúc báo cáo	3
2 Cơ sở lý thuyết và công nghệ liên quan	4
2.1 Điện toán biên và Edge AI	4
2.2 Hệ multi-agent và mô hình điều phối	4
2.3 MQTT trong hệ IoT/Edge	5
2.4 Phát hiện bất thường, Isolation Forest và ONNX	5
2.5 Đo lường hiệu năng trên biên	6
2.6 Tóm tắt chương	6
3 Phân tích yêu cầu và lựa chọn giải pháp	7
3.1 Yêu cầu từ đề bài	7
3.2 Lựa chọn miền ứng dụng	8
3.3 Yêu cầu chức năng	8
3.4 Yêu cầu phi chức năng	8
3.5 Các phương án và quyết định thiết kế	9
3.6 Tóm tắt chương	10
4 Thiết kế hệ thống	11
4.1 Kiến trúc tổng thể	11
4.2 Vai trò từng agent	11
4.3 Luồng xử lý end-to-end	12

4.4	Thiết kế giao thức MQTT và schema	12
4.5	Thiết kế mô hình AI cục bộ	14
4.6	Chính sách quyết định và ngưỡng fallback	14
4.7	Thiết kế đo lường và quan sát	15
4.8	Tóm tắt chương	15
5	Hiện thực và triển khai	16
5.1	Môi trường công nghệ	16
5.2	Cấu trúc triển khai	16
5.3	Hiện thực các agent	17
5.3.1	Sensor Agent	17
5.3.2	Analysis Agent	17
5.3.3	Decision Agent	17
5.4	Huấn luyện và đóng gói model	17
5.5	Triển khai Docker và giới hạn tài nguyên	18
5.6	Công cụ kiểm thử và fault injection	18
5.7	Quy trình tái lập (tóm tắt)	18
5.8	Tóm tắt chương	19
6	Thực nghiệm và đánh giá	20
6.1	Mục tiêu và giả thuyết thực nghiệm	20
6.2	Môi trường và phương pháp đo	20
6.3	Kết quả tài nguyên	21
6.4	Kết quả độ trễ và throughput	21
6.5	Kết quả kịch bản lỗi	22
6.6	Phân tích trade-off	23
6.7	Đối chiếu với mục tiêu đề bài	23
6.8	Thảo luận hạn chế thực nghiệm	24
6.9	Tóm tắt chương	24
7	Kết luận và hướng phát triển	25
7.1	Kết luận	25
7.2	Hướng phát triển	25
7.3	Bài học kinh nghiệm	26

Danh sách hình vẽ

4.1	Kiến trúc logic hệ multi-agent trên edge	11
4.2	Sơ đồ tuần tự — luồng E2E bình thường	13
4.3	Sơ đồ tuần tự — Analysis timeout và chế độ degraded	13
6.1	So sánh phân vị độ trễ E2E và inference ONNX (cửa sổ 40 s)	22
6.2	RAM quan sát theo thành phần (mẫu <code>docker stats</code> cuối cửa sổ)	22

Danh sách bảng

2.1	Hai mô hình điều phối phổ biến và ý nghĩa với đồ án	5
3.1	Đối chiếu yêu cầu đề bài và giải pháp	7
3.2	Quyết định thiết kế chính	9
4.1	Phân công trách nhiệm	12
4.2	Topics MQTT	13
4.3	Thông số mô hình	14
5.1	Stack công nghệ	16
5.2	Nhóm kiểm thử phục vụ tái lập thí nghiệm	18
6.1	Cấu hình thí nghiệm (cửa sổ ổn định)	20
6.2	Mẫu <code>docker stats</code> cuối cửa sổ đo	21
6.3	Tổng hợp kết quả thực nghiệm (Docker Compose, mỗi agent 2vCPU / 4GB, CPU-only; cửa sổ 40s)	21
6.4	Fault path — dừng Analysis Agent	23
6.5	Phân tích trade-off	23
6.6	Đối chiếu mục tiêu đề bài	23

Danh mục từ viết tắt

Viết tắt	Diễn giải
AI	Artificial Intelligence — Trí tuệ nhân tạo
CPU	Central Processing Unit
E2E	End-to-End — Từ đầu đến cuối pipeline
IoT	Internet of Things
JSON	JavaScript Object Notation
LLM	Large Language Model
MQTT	Message Queuing Telemetry Transport
ONNX	Open Neural Network Exchange
p50 / p95	Phân vị thứ 50 / 95 của phân phối độ trễ
QoS	Quality of Service
RAM	Random Access Memory
RSS	Resident Set Size
VM	Virtual Machine
WS	WebSocket

Chương 1

Mở đầu

1.1 Đặt vấn đề

Sự bùng nổ của IoT và các hệ giám sát liên tục (môi trường, nhà máy, đô thị thông minh) tạo ra lượng dữ liệu cảm biến lớn ngay tại hiện trường. Nếu mọi mẫu đo đều được đẩy thô lên đám mây để suy luận rồi mới trả quyết định, hệ thống dễ gặp ba hạn chế thực tế: (i) độ trễ tăng theo điều kiện mạng; (ii) phụ thuộc băng thông và chi phí truyền tải; (iii) mất khả năng phản ứng khi liên kết đám mây gián đoạn. **Điện toán biên** giải quyết một phần các hạn chế này bằng cách đặt suy luận và điều phối gần nguồn dữ liệu [9].

Trong nhiều kịch bản biên, bài toán không còn là “một tiến trình đơn lẻ chạy model”, mà là phối hợp nhiều thành phần chuyên trách: thu thập, phân tích, ra quyết định. Cách tổ chức này khớp với tư duy **multi-agent**: mỗi agent có vai trò rõ, trạng thái riêng và giao tiếp qua mạng. Tuy nhiên, nút biên thường bị ràng buộc mạnh về CPU, RAM và không có GPU. Do đó, lựa chọn giao thức nhẹ, mô hình AI đủ nhỏ, và cơ chế chịu lỗi khi một agent mất phản hồi trở thành điều kiện sống còn — không phải phần “làm đẹp” sau khi demo chạy được.

Đề tài học phần CE2206 yêu cầu hiện thực hóa các năng lực trên một cách **định lượng** [1]: không chỉ chứng minh hệ “chạy được”, mà phải đo tài nguyên, độ trễ (p50/p95), throughput, độ trễ end-to-end (E2E), và chứng minh ít nhất một kịch bản lỗi được xử lý an toàn (hệ không sập toàn bộ).

Từ đó, đề án đặt ra các câu hỏi nghiên cứu/triển khai cụ thể:

1. Làm thế nào để tổ chức ≥ 3 agent vai trò khác nhau trên môi trường edge mô phỏng (2vCPU / 4GB / CPU-only mỗi agent) mà vẫn giao tiếp **thật qua mạng**?
2. AI cục bộ nào đủ nhẹ để chạy ổn định trong ngân sách 4GB, đồng thời cho độ trễ suy luận ở mức mili-giây?
3. Khi agent phân tích mất phản hồi, orchestrator có thể tiếp tục ra quyết định ở chế độ suy giảm (*degraded*) như thế nào?

4. Trade-off giữa độ trễ, tài nguyên và độ phức tạp mô hình thể hiện ra sao trên số liệu đo được?

1.2 Mục tiêu đồ án

Đồ án hướng tới các mục tiêu cụ thể sau:

1. **Thiết kế và triển khai** hệ multi-agent (≥ 3 agent, vai trò khác nhau) trên môi trường edge mô phỏng bằng container/VM, mỗi agent giới hạn **2 vCPU / 4 GB RAM, CPU-only**.
2. **Đảm bảo giao tiếp thật qua mạng** bằng MQTT (Eclipse Mosquitto), với schema JSON thống nhất; cấm giả lập bằng gọi hàm nội bộ hay shared memory giữa các agent.
3. **Tích hợp suy luận AI cục bộ** trên Analysis Agent, ưu tiên mô hình tối ưu cho biên (Isolation Forest xuất ONNX, chạy ONNX Runtime CPU).
4. **Xây dựng điều phối tập trung nhẹ** tại Decision Agent, kèm xử lý timeout Analysis $\rightarrow$ fallback ngưỡng cứng + trạng thái **degraded**.
5. **Đánh giá định lượng** peak RAM/CPU, latency p50/p95, throughput và E2E; cung cấp số liệu tái lập được (`reports/metrics_run.json`).
6. **Bổ sung quan sát vận hành** qua dashboard realtime phục vụ demo và giám sát, nhưng không tham gia vòng quyết định nghiệp vụ.

1.3 Phạm vi nghiên cứu

Trong phạm vi.

- Miền giám sát môi trường thông minh với bốn đại lượng: nhiệt độ, độ ẩm, PM2.5, CO₂; dữ liệu cảm biến **giả lập có kiểm soát** để tái lập thí nghiệm.
- Ba agent nghiệp vụ (Sensor, Analysis, Decision) + MQTT broker + dashboard quan sát (tùy chọn khi demo).
- Mô hình anomaly detection cổ điển xuất ONNX; LLM nhỏ Q4 chỉ được xem là hướng mở rộng/điểm cộng, không phải đường chính của bản nộp.
- Triển khai Docker Compose với resource limit tương đương đề bài (`cpus: 2.0, mem_limit: 4g`).

Ngoài phạm vi.

- Cảm biến phần cứng thật và hiệu chuẩn môi trường thực địa.
- So sánh đầy đủ nhiều mức lượng tử hóa LLM / pruning trên cùng metric.

- Đo năng lượng tiêu thụ và chi phí vận hành dài hạn trên thiết bị biên vật lý.
- Hybrid edge–cloud production (chỉ thảo luận định hướng ở Chương 7).

1.4 Phương pháp thực hiện

Đồ án kết hợp phân tích yêu cầu, thiết kế kiến trúc, hiện thực tăng dần và thực nghiệm có kiểm soát, theo các giai đoạn:

1. **Phân tích đề bài:** chuyển ràng buộc cứng (agent, tài nguyên, MQTT, AI, fault, metrics) thành yêu cầu chức năng/phi chức năng và tiêu chí chấp nhận.
2. **Thiết kế:** pipeline Sensor → Analysis → Decision; schema/`trace_id`; chính sách quyết định và fallback; kế hoạch đo E2E.
3. **Hiện thực tăng dần:** Sensor MQTT → Analysis ONNX → Decision + timeout/fallback → Docker limit → script verify/metrics/fault → dashboard.
4. **Thực nghiệm:** thu thập metrics cửa sổ cố định; inject lỗi Analysis; đối chiếu số liệu với mục tiêu đề và thảo luận trade-off/hạn chế.

1.5 Đóng góp chính

1. Một hệ multi-agent biên **tái lập được**, tuân thủ ngân sách 2 vCPU/4 GB và giao tiếp MQTT thật giữa các process/container tách biệt.
2. Pipeline AI cục bộ ONNX gọn (~1,16 MB model) với độ trễ inference mili-giây trên CPU, tách biệt pha huấn luyện và pha suy luận trên biên.
3. Cơ chế **fault tolerance** có thể demo: timeout Analysis → alert **degraded**, các agent còn lại tiếp tục sống.
4. Bộ số liệu E2E/inference/tài nguyên kèm dashboard realtime hỗ trợ đánh giá và trình diễn.

1.6 Cấu trúc báo cáo

Báo cáo gồm bảy chương. Chương 2 trình bày cơ sở lý thuyết và công nghệ liên quan. Chương 3 phân tích yêu cầu và lý do lựa chọn miền/giải pháp. Chương 4 mô tả thiết kế kiến trúc, giao thức, mô hình AI và chính sách quyết định. Chương 5 trình bày hiện thực và triển khai. Chương 6 báo cáo thực nghiệm, phân tích số liệu và trade-off. Chương 7 kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết và công nghệ liên quan

2.1 Điện toán biên và Edge AI

Điện toán biên đẩy một phần tính toán, lưu trữ và suy luận ra khỏi trung tâm dữ liệu truyền thống, tiến gần thiết bị đầu cuối [9]. Khác với mô hình “mọi thứ lên cloud”, biên nhằm tối ưu hoá vị trí xử lý theo ràng buộc độ trễ, băng thông và tính sẵn sàng cục bộ. Trong phổ triển khai, có thể hình dung đám mây (tính toán lớn, độ trễ cao hơn), fog/edge gần hiện trường (tính toán vừa đủ, độ trễ thấp), và thiết bị đầu cuối (cảm biến/actuator).

Edge AI nhấn mạnh việc chạy mô hình học máy ngay trên nút biên nhằm: giảm độ trễ phản hồi; giảm lưu lượng đẩy thô lên đám mây; và duy trì khả năng suy luận khi liên kết đám mây không ổn định. Điểm mấu chốt kỹ thuật là nút biên thường bị ràng buộc mạnh về CPU, RAM, năng lượng và thiếu GPU. Do đó, lựa chọn mô hình nhỏ, runtime nhẹ và **đo lường tài nguyên dưới đúng hạn mức** là trọng tâm — không phải phần phụ sau khi hoàn thiện chức năng.

Với đồ án CE2206, “edge” được mô phỏng bằng container/VM có giới hạn 2vCPU / 4 GB / CPU-only cho mỗi agent. Cách làm này không thay thế thiết bị biên vật lý, nhưng đủ để kiểm chứng kiến trúc, giao tiếp và trade-off hiệu năng trong điều kiện tài nguyên bị siết tương đương đề bài.

2.2 Hệ multi-agent và mô hình điều phối

Một **agent** là thực thể phần mềm có mục tiêu, trạng thái nội bộ và khả năng tương tác với môi trường cũng như agent khác. Hệ multi-agent phân chia trách nhiệm (chuyên môn hóa), đồng thời đặt ra hai bài toán then chốt: *coordination* (ai làm gì, theo thứ tự/luồng nào) và *fault tolerance* (hệ ứng xử ra sao khi một agent lỗi).

Đồ án chọn **điều phối tập trung nhẹ**: Decision Agent là orchestrator, nhận reading/result

Bảng 2.1: Hai mô hình điều phối phổ biến và ý nghĩa với đồ án

Mô hình	Đặc điểm	Phù hợp khi
Tập trung	Một orchestrator tổng hợp thông tin và ra quyết định cuối	Pipeline rõ ràng, cần điểm kiểm soát thống nhất, quy mô vừa
Phi tập trung	Các agent đàm phán / đồng thuận ngang hàng	Hệ lớn, cần tránh single point of failure mạnh

qua MQTT và phát alert. “Nhẹ” ở đây có nghĩa là không kéo theo framework orchestration nặng (CrewAI, LangGraph, AutoGen) — các framework này tiện cho demo trên máy mạnh nhưng dễ đội RAM và làm mờ ranh giới process trên biên. Thay vào đó, vòng đời điều phối được hiện thực trực tiếp bằng subscribe MQTT, bảng `PendingTrace` và timer timeout.

2.3 MQTT trong hệ IoT/Edge

MQTT là giao thức pub/sub nhẹ, phổ biến trong IoT [7]. Client publish bản tin lên *topic*; broker (Eclipse Mosquitto [3]) chuyển tới các subscriber của topic đó. So với polling HTTP hoặc RPC đồng bộ chặt, MQTT phù hợp biên vì: overhead thấp; tách rời producer/consumer theo không gian và thời gian; và hỗ trợ mức QoS.

Đồ án dùng MQTT v3.1.1 qua thư viện Paho [4], QoS 1 cho luồng nghiệp vụ (readings/results/alerts/l) nhằm giảm mất tin mà không bắt buộc QoS 2 (chi phí bắt tay cao hơn, ít cần thiết với tần số cảm biến vài giây/lần). Broker đóng vai trò trung gian duy nhất cho giao tiếp liên agent: mọi trao đổi đều là message JSON trên topic, không có gọi hàm chéo process.

2.4 Phát hiện bất thường, Isolation Forest và ONNX

Giám sát môi trường thường cần phát hiện mẫu “lạ” so với nền vận hành bình thường (nhiệt độ/PM2.5/CO₂ bất thường), hơn là phân loại có nhãn đầy đủ theo kiểu học có giám sát. **Isolation Forest** [5] là thuật toán học không giám sát dựa trên ý tưởng: điểm bất thường dễ bị *cô lập* sớm hơn trong một tập hợp cây phân hoạch ngẫu nhiên (*isolation trees*). Mỗi cây chọn ngẫu nhiên một đặc trưng và một ngưỡng cắt; điểm nằm ở vùng thừa thường rơi vào lá sau ít lần cắt hơn, nên độ dài đường đi trung bình ngắn hơn. Điểm số bất thường được suy từ độ dài đường đi này: càng ngắn càng “lạ”.

Trong đồ án, các tham số chính gồm số cây $n = 100$ và tỷ lệ contamination 0,05 (kỳ vọng tỷ lệ bất thường khi hiệu chỉnh ngưỡng). Đặc trưng đầu vào là bốn đại lượng cảm biến đã chuẩn hóa để tránh thang đo lệch (ví dụ CO₂ vài trăm–nghìn ppm so với nhiệt độ vài chục °C). Thuật toán phù hợp dữ liệu số nhiều chiều với chi phí suy luận thấp — khớp ràng buộc CPU-only trên biên.

Pipeline thực tế gồm: huấn luyện scaler + Isolation Forest trên dữ liệu synthetic “normal”, rồi xuất toàn pipeline sang định dạng **ONNX**. ONNX là định dạng trung gian cho mô hình học máy; tại runtime biên, Analysis Agent chỉ cần ONNX Runtime [6] trên CPU, không mang theo stack huấn luyện nặng (scikit-learn đầy đủ chỉ cần ở pha train). Cách tách này giúp image nhỏ hơn, suy luận ổn định hơn, và dễ đóng gói trong Docker [2].

2.5 Đo lường hiệu năng trên biên

Đề bài yêu cầu đánh giá định lượng, không chỉ mô tả định tính. Các chỉ số then chốt gồm:

- **Tài nguyên:** peak RAM (RSS) và CPU trung bình/đỉnh, đo dưới đúng `mem_limit/cpus`.
- **Độ trễ:** phân vị p50 và p95 — ổn định hơn so với chỉ dùng trung bình khi phân phối có đuôi dài.
- **Throughput:** số message (reading/result/alert) trên đơn vị thời gian.
- **E2E latency:** từ sự kiện gốc gắn với `SensorReading` đến `DecisionAlert` cùng `trace_id`.

Việc đo phải gắn với điều kiện chạy thật (container có limit). Đo trên máy mạnh không giới hạn rồi suy luận về khả năng biên dễ dẫn đến kết luận sai. Trong đồ án, E2E và inference được thu thập tự động qua công cụ đo subscribe MQTT; tài nguyên container được lấy từ `docker stats`.

2.6 Tóm tắt chương

Chương này thiết lập nền tảng: Edge AI đòi hỏi tối ưu tài nguyên; multi-agent cần giao thức và điều phối rõ; MQTT/ONNX phù hợp ràng buộc biên; metrics p50/p95 và E2E là chuẩn đánh giá bắt buộc. Các lựa chọn công nghệ cụ thể sẽ được biện luận tiếp ở Chương 3 và hiện thực ở các chương sau.

Chương 3

Phân tích yêu cầu và lựa chọn giải pháp

3.1 Yêu cầu từ đề bài

Bảng 3.1 tóm tắt ràng buộc cứng của đề và cách đề án đáp ứng [1]. Ba năng lực được nhấn mạnh khi chấm điểm là: (1) tối ưu tài nguyên; (2) phối hợp phân tán có xử lý lỗi; (3) đánh giá định lượng trade-off.

Bảng 3.1: Đối chiếu yêu cầu đề bài và giải pháp

Hạng mục	Yêu cầu	Giải pháp đề án
Số agent	≥ 3 , vai trò khác nhau	Sensor, Analysis, Decision
Phần cứng	2vCPU, 4GB, CPU-only / agent	Docker <code>cpus: 2.0, mem_limit: 4g</code>
Giao tiếp	Qua mạng thật	MQTT Mosquitto + schema JSON
AI cục bộ	≥ 1 agent inference trên edge	Analysis + ONNX Runtime
Điều phối	Mô tả rõ tập trung/phi tập trung	Tập trung nhẹ tại Decision
Fault	≥ 1 scenario, hệ không sập toàn bộ	Analysis timeout $\rightarrow$ fallback <code>degraded</code>
Metrics	RAM/CPU, latency p50/p95, throughput, E2E	Công cụ đo + <code>metrics_run.json</code>
Sản phẩm báo cáo	Source tái lập + báo cáo kỹ thuật	Repo + báo cáo này

3.2 Lựa chọn miền ứng dụng

Đề gợi ý nhiều hướng (vision, RAG, predictive maintenance, giám sát môi trường...). Nhóm chọn **giám sát môi trường thông minh** vì các lý do sau.

Thứ nhất, miền khớp ví dụ minh họa trong đề và cho phép tách bạch ba vai trò: thu thập cảm biến, phân tích bất thường, ra quyết định cảnh báo. Thứ hai, dữ liệu nhiệt độ/độ ẩm/PM2.5/CO₂ dễ giả lập có kiểm soát, nhờ đó thí nghiệm **tái lập được** — điều kiện cần để báo cáo số liệu thuyết phục. Thứ ba, bài toán anomaly detection cổ điển vừa khung 4 GB RAM; vẫn còn dư địa kỹ thuật để mở LLM nhỏ Q4 nếu cần điểm cộng sau này. Thứ tư, dễ inject lỗi (ngắt Analysis) để demo fault handling mà không cần hạ tầng phức tạp.

Listing 3.1: Pipeline nghiệp vụ của miền giám sát môi trường

Sensor Agent -> Analysis Agent (AI local) -> Decision Agent
(thu thập) (phat hien bat thuong) (canh bao / hanh dong)

3.3 Yêu cầu chức năng

Các yêu cầu chức năng được viết ở mức có thể kiểm chứng bằng chạy hệ và script verify:

RF-01. Sensor định kỳ phát bản tin cảm biến chuẩn hóa lên MQTT (chu kỳ mặc định 2s).

RF-02. Analysis nhận reading, chạy inference cục bộ, phát **AnalysisResult** (score, label, inference_ms).

RF-03. Decision nhận kết quả, áp chính sách cảnh báo, phát **DecisionAlert**.

RF-04. Decision theo dõi timeout theo **trace_id**; hết hạn thì fallback threshold và đánh dấu **degraded=true**.

RF-05. Mọi agent phát heartbeat; mọi message nghiệp vụ mang **trace_id**.

RF-06. Message JSON sai schema bị từ chối/ghi log, **không** làm crash process.

RF-07. (Mở rộng) Dashboard subscribe MQTT và hiển thị trạng thái, latency, alert realtime.

3.4 Yêu cầu phi chức năng

RNF-01. Tài nguyên. Mỗi agent ≤ 2 vCPU, ≤ 4 GB RAM, không GPU; không nói limit chỉ để “cho chạy được”.

RNF-02. Độ trễ. E2E ở mức mili-giây khi ổn định (mục tiêu thực nghiệm: p50 <

100 ms).

RNF-03. Khả năng sống sót. Mất Analysis không làm dừng Sensor/Decision; hệ vẫn phát quyết định (degraded).

RNF-04. Tái lập. Có script deploy, đo metrics, inject fault; số liệu gắn với file kết quả.

RNF-05. Tách biệt tiến trình. Cắm gộp ba agent một process để demo nhanh.

3.5 Các phương án và quyết định thiết kế

Các quyết định chính được tóm tắt ở Bảng 3.2. Phần dưới làm rõ lý do đằng sau vài lựa chọn then chốt.

MQTT thay vì gRPC/HTTP sync. Pub/sub khớp luồng “một reading nhiều người nghe” (Analysis, Decision, Dashboard). HTTP sync buộc caller chờ callee; khi Analysis chậm/mất, mô hình request–response dễ kéo theo timeout chồng chéo và coupling chặt hơn.

Orchestrator tự viết thay vì framework nặng. Framework multi-agent hiện đại mạnh về lập trình tác vụ/LLM, nhưng thường kéo theo dependency lớn và mô hình tiến trình khác với “1 agent / 1 VM”. Trên trần 4 GB, rủi ro vượt ngân sách và khó giải thích rõ luồng MQTT theo đúng đề là đáng kể.

Isolation Forest/ONNX thay vì LLM ngay từ đầu. LLM 0,5B–3B (kể cả Q4) có thể vượt hoặc tiệm cận trần RAM tùy runtime; độ trễ cũng khó giữ mức vài mili-giây trên CPU. Isolation Forest + ONNX cho phép chứng minh AI cục bộ đúng đề trước, rồi mới cân nhắc LLM như điểm cộng có số liệu đối chứng.

Bảng 3.2: Quyết định thiết kế chính

Hạng mục	Chọn	Loại / trì hoãn	Lý do
Giao tiếp	MQTT	gRPC/HTTP sync	Phù hợp IoT, nhẹ, pub/sub
Điều phối Model	Tập trung nhẹ Isolation Forest → ONNX	Framework nặng LLM 3B Q4 ngay	Đủ rõ, ít RAM An toàn trần 4 GB
Runtime	ONNX Runtime CPU	PyTorch full	Nhẹ, đủ inference
Deploy	Docker Compose limit	Nhiều process/1 container	1 agent / 1 VM
Fallback	Threshold rules	Chỉ log im lặng	Vẫn ra quyết định

3.6 Tóm tắt chương

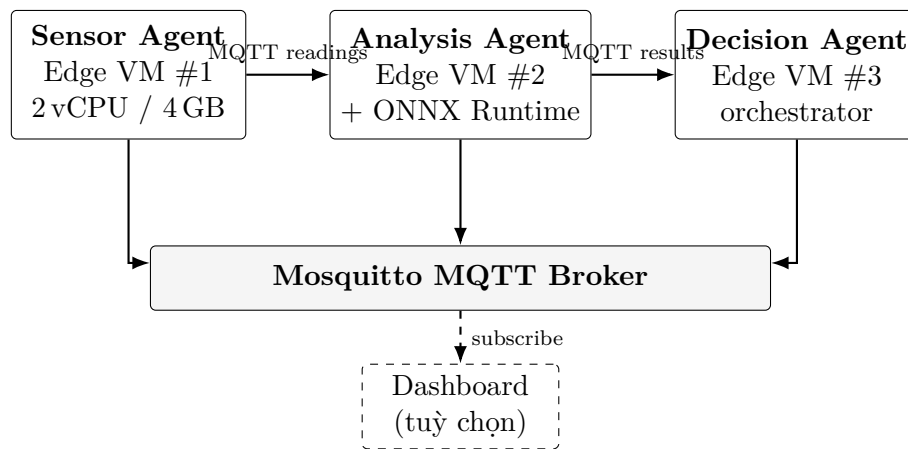
Yêu cầu đề bài đã được chuyển thành RF/RNF kiểm chứng được. Miền giám sát môi trường và stack MQTT + ONNX + orchestrator tập trung nhẹ được chọn có chủ đích để cân bằng tính đúng đề, khả năng chạy trong 4 GB, và đo được số liệu.

Chương 4

Thiết kế hệ thống

4.1 Kiến trúc tổng thể

Hình 4.1 mô tả kiến trúc logic. Ba agent edge giao tiếp qua Mosquitto; dashboard chỉ quan sát (subscribe), không tham gia vòng quyết định. Mỗi agent là một container/process riêng với hạn mức 2 vCPU / 4 GB. Phần dùng chung chỉ gồm schema message, cấu hình và tiện ích đo lường — không chứa logic điều phối và không cho phép các agent gọi nhau như thư viện nội bộ.



Hình 4.1: Kiến trúc logic hệ multi-agent trên edge

Thiết kế này phản ánh đúng tinh thần đề: phân tán về tiến trình, tập trung nhẹ về điều phối, và mọi tương tác liên agent đều quan sát được trên MQTT.

4.2 Vai trò từng agent

Ranh giới trách nhiệm giúp tránh “agent thần thánh” ôm hết việc, đồng thời làm rõ điểm đo: Analysis chịu trách nhiệm độ trễ inference; Decision chịu trách nhiệm E2E và degraded mode.

Bảng 4.1: Phân công trách nhiệm

Agent	Được phép	Không được phép
sensor_agent	Giả lập/chuẩn hóa cảm biến; publish reading + heartbeat	Chạy model AI; ra quyết định cảnh báo
analysis_agent	Subscribe reading; inference cục bộ; publish result	Điều phối toàn hệ; bỏ qua MQTT
decision_agent	Policy/alert; timeout fallback; heartbeat/control	Thay thế hoàn toàn AI chính (chỉ fallback khi degraded)
dashboard	Quan sát MQTT, hiển thị metrics/alert	Tham gia vòng quyết định nghiệp vụ

4.3 Luồng xử lý end-to-end

Luồng bình thường.

1. Sensor tạo `SensorReading` (có `trace_id`, `ts`) và publish lên `edge/sensor/readings`.
2. Analysis parse reading, chạy ONNX, đo `inference_ms`, publish `AnalysisResult` cùng `trace_id`.
3. Decision map kết quả sang `DecisionAlert` (`severity`, `action`, `reason`, `e2e_latency_ms`).
4. Dashboard (nếu bật) nhận và hiển thị realtime phục vụ quan sát.

Luồng khi Analysis không phản hồi.

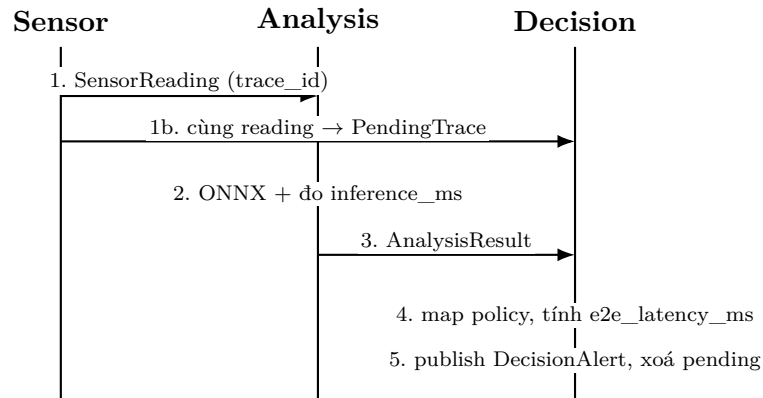
1. Decision lưu `PendingTrace` khi nhận reading (theo `trace_id`).
2. Hết `analysis_timeout_sec` (mặc định 5 s) mà chưa có result → gọi `threshold_fallback`.
3. Publish alert với `severity=degraded`, `degraded=true`.
4. Sensor và Decision tiếp tục chạy; Analysis có thể được khởi động lại sau.
5. Result đến muộn sau khi đã fallback được **bỏ qua** để tránh double-alert cho cùng `trace_id`.

Cơ chế trên hiện thực yêu cầu “hệ không sập toàn bộ”: mất một mắt xích AI vẫn còn đường ra quyết định (suy giảm chất lượng), đồng thời làm rõ trade-off độ trễ khi lỗi (E2E bị chi phối bởi timeout).

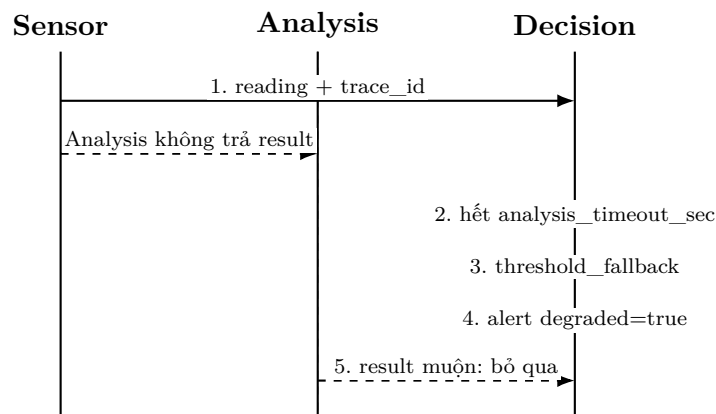
Hình 4.2 và Hình 4.3 tóm tắt hai luồng dưới dạng sơ đồ tuần tự.

4.4 Thiết kế giao thức MQTT và schema

Nguyên tắc schema message:



Hình 4.2: Sơ đồ tuần tự — luồng E2E bình thường



Hình 4.3: Sơ đồ tuần tự — Analysis timeout và chế độ degraded

Bảng 4.2: Topics MQTT

Topic	Publisher	Subscriber
edge/sensor/readings	sensor	analysis, decision, dashboard
edge/analysis/results	analysis	decision, dashboard
edge/decision/alerts	decision	dashboard / log
edge/system/heartbeat	mọi agent	decision, dashboard
edge/system/control	decision	sensor, analysis

- Payload JSON UTF-8; các kiểu tương ứng `SensorReading`, `AnalysisResult`, `DecisionAlert`, `Heartbeat`.
- Field bắt buộc được kiểm tra khi nhận; thiếu field thì từ chối bản tin, vòng lặp agent vẫn sống.
- `trace_id` là khóa nối E2E giữa `reading` → `result` → `alert`.
- QoS 1 cho luồng nghiệp vụ.

Decision subscribe cả readings và results: readings dùng để mở `PendingTrace` và làm đầu vào fallback; results dùng để quyết định theo AI khi còn kịp trước timeout.

4.5 Thiết kế mô hình AI cục bộ

Bảng 4.3: Thông số mô hình

Thuộc tính	Giá trị
Thuật toán	Isolation Forest + StandardScaler
Thư viện huấn luyện	scikit-learn [8]
Xuất mô hình	skl2onnx → ONNX (~1,16 MB)
Runtime suy luận	ONNX Runtime (CPU)
Đặc trưng	<code>temperature_c</code> , <code>humidity_pct</code> , <code>pm25_ugm3</code> , <code>co2_ppm</code>
Huấn luyện	2000 mẫu normal; contamination = 0,05; 100 cây
Kiểm tra synthetic	Normal $\approx 96\%$; anomaly $\approx 100\%$

Lựa chọn ưu tiên độ nhỏ và độ trễ thấp hơn độ phức tạp biểu diễn. Kết quả kiểm tra nhanh trên synthetic chỉ mang tính *smoke test* của pipeline train/export; không được suy diễn thành độ chính xác trên dữ liệu thực địa. LLM nhỏ (ví dụ Qwen2.5-0.5B Q4) là hướng đi tiềm năng có kiểm soát tài nguyên, không phải đường chính của bản nộp.

4.6 Chính sách quyết định và ngưỡng fallback

Khi có kết quả AI kịp thời:

- `is_anomaly=false` → `severity=info`, `action=continue_monitoring`.
- `is_anomaly=true` → `action=raise_alert`; critical nếu `score` $\geq 0,05$, ngược lại `warning`.

Khi timeout, Decision áp ngưỡng cứng từ cấu hình:

- Nhiệt độ $\geq 40^\circ\text{C}$;
- $\text{PM}_{2.5} \geq 75 \mu\text{g}/\text{m}^3$;
- $\text{CO}_2 \geq 1500 \text{ ppm}$.

Vượt ngưỡng → alert degraded kèm lý do chi tiết; không vượt → vẫn phát bản tin degraded ghi nhận timeout nhưng ngưỡng OK. Cách làm này bảo đảm luôn có quyết định quan sát được trên topic alerts, kể cả khi AI im lặng.

4.7 Thiết kế đo lường và quan sát

Hệ có hai lớp đo:

- **Trong agent:** MetricsCollector (psutil) ghi mẫu theo chu kỳ cấu hình.
- **Ngoài agent:** công cụ đo subscribe MQTT, ghép bản tin theo `trace_id`, tính p50/p95 E2E & inference, kết hợp `docker stats`.

Dashboard (Flask + Flask-SocketIO + Chart.js) subscribe các topic và đẩy WebSocket tới trình duyệt để demo trực quan. Dashboard là lớp quan sát, không thay thế file metrics phục vụ báo cáo.

4.8 Tóm tắt chương

Kiến trúc ba agent + MQTT + orchestrator tập trung nhẹ bảo đảm đúng ràng buộc đề; schema/`trace_id` phục vụ E2E; ONNX phục vụ AI biên; timeout/fallback phục vụ fault tolerance; metrics/dashboard phục vụ đánh giá và demo.

Chương 5

Hiện thực và triển khai

5.1 Môi trường công nghệ

Bảng 5.1: Stack công nghệ

Thành phần	Công nghệ
Ngôn ngữ	Python 3.11
Broker	Eclipse Mosquitto 2
MQTT client	paho-mqtt [4]
AI	scikit-learn, skl2onnx, onnxruntime
Đóng gói	Docker / Docker Compose [2]
Quan sát	Flask, Flask-SocketIO, Chart.js
Đo tài nguyên	psutil, docker stats

Stack được chọn theo tiêu chí “đủ làm đúng đề trên biên”: Python phổ biến cho IoT/AI nhẹ; Mosquitto ổn định; ONNX Runtime cho inference CPU; Compose để thiết `cpus/mem_limit` thống nhất giữa các máy.

5.2 Cấu trúc triển khai

Mã nguồn được tổ chức theo nguyên tắc **một agent — một tiến trình**: thư mục riêng cho Sensor, Analysis, Decision; broker MQTT; cấu hình chung; model ONNX; công cụ đo/verify/fault; và dashboard quan sát. Quy ước phát triển nhấn mạnh: không gọi chéo giữa agent, không nói resource limit để “cho chạy được”, message sai schema không được làm sập process, mọi luồng nghiệp vụ mang `trace_id`, Decision phải có timeout/fallback.

5.3 Hiện thực các agent

5.3.1 Sensor Agent

Sensor đọc chu kỳ lấy mẫu (mặc định 2s) và tỷ lệ sinh mẫu bất thường giả lập, tạo vector cảm biến, gán `trace_id` cùng dấu thời gian, rồi publish QoS 1 lên topic readings. Định kỳ gửi heartbeat để lớp quan sát biết agent còn sống. Sensor **không** chạy model và **không** tự cảnh báo — đúng ranh giới trách nhiệm.

5.3.2 Analysis Agent

Analysis nạp mô hình ONNX lúc khởi động, subscribe readings và kiểm tra schema khi nhận bản tin. Với mỗi mẫu hợp lệ: đo thời gian suy luận, sinh kết quả (score, nhãn bất thường) và publish. Bản tin malformed được ghi nhận rồi bỏ qua; vòng lặp MQTT không bị phá. Thiết kế này cô lập rủi ro dữ liệu bản khởi rủi ro sập process.

5.3.3 Decision Agent

Decision subscribe readings, results và heartbeat. Khi có reading, mở bản ghi chờ theo `trace_id`. Vòng kiểm tra timeout định kỳ thực hiện logic rút gọn như sau:

Listing 5.1: Gia ma — timeout và fallback tại Decision

```
voi moi pending[trace_id]:
    neu (now - received_at) < timeout: tiep tuc
    nguoc lai:
        ap dung nguong fallback tren reading
        phat DecisionAlert (degraded=true, e2e_latency)
        xoa pending[trace_id] # result den muon se bi bo qua
```

Khi result đến đúng hạn, Decision map sang mức cảnh báo/hành động theo kết quả AI, tính `e2e_latency_ms`, rồi xoá pending tương ứng.

5.4 Huấn luyện và đóng gói model

Pha huấn luyện sinh dữ liệu synthetic, huấn luyện scaler + Isolation Forest, xuất ONNX và metadata. Khi build image triển khai, model được chuẩn bị sẵn để Analysis luôn có file suy luận trên mọi máy, tránh phụ thuộc thao tác thủ công. Pha train dùng scikit-learn; pha serve trên biên chỉ cần ONNX Runtime.

5.5 Triển khai Docker và giới hạn tài nguyên

Mỗi service agent trong Compose khai báo giới hạn:

Listing 5.2: Giới hạn tài nguyên mỗi agent

```
cpus: 2.0
mem_limit: 4g
memswap_limit: 4g
```

Không gắn GPU. Broker Mosquitto dùng image chính thức; các agent dùng image chung với `command` khác nhau để khởi động đúng entrypoint. Dashboard giới hạn nhẹ hơn (quan sát, không tính là agent đề bài): khoảng 0,5 CPU / 256 MB; trên macOS, cổng host thường map **5001** vì cổng 5000 hay bị hệ điều hành chiếm.

Việc siết limit ngay từ Compose buộc mọi số liệu tài nguyên trong Chương 6 phản ánh đúng điều kiện đề, thay vì phản ánh máy host “thoải mái”.

5.6 Công cụ kiểm thử và fault injection

Bảng 5.2: Nhóm kiểm thử phục vụ tái lập thí nghiệm

Nhóm	Mục đích
Verify E2E	Chứng minh pipeline reading → result → alert
Fault injection	Dùng Analysis để kích hoạt chế độ degraded
Verify degraded	Assert alert có <code>degraded=true</code> sau timeout
Thu thập metrics	Ghi p50/p95, throughput, <code>docker stats</code> ra file kết quả

Các bước trên biến RF/RNF thành kiểm thử vận hành lặp lại được: không chỉ “nhìn log thấy có vẻ đúng”, mà kiểm chứng được số alert, cờ `degraded`, và latency.

5.7 Quy trình tái lập (tóm tắt)

1. Khởi động toàn bộ service với resource limit đúng đề.
2. Chạy verify pipeline E2E ở chế độ ổn định.
3. Thu thập metrics trong cửa sổ thời gian cố định.
4. Inject lỗi Analysis, verify alert degraded, rồi khởi động lại Analysis.

Quy trình này bảo đảm thí nghiệm **tái lập được** và gắn trực tiếp với số liệu Chương 6.

5.8 Tóm tắt chương

Hiện thực bám sát thiết kế: ba tiến trình tách biệt, MQTT + schema, ONNX cục bộ, timeout/fallback, resource limit đúng đề, kèm verify/metrics/fault và dashboard quan sát.

Chương 6

Thực nghiệm và đánh giá

6.1 Mục tiêu và giả thuyết thực nghiệm

Thực nghiệm nhằm trả lời bốn câu hỏi, tương ứng giả thuyết làm việc:

1. **H1 — Tài nguyên:** Hệ chạy ổn định trong giới hạn 2 vCPU/4 GB/CPU-only; RAM đỉnh mỗi agent $\ll$ 4 GB.
2. **H2 — Độ trễ:** E2E và inference ở mức chấp nhận được cho giám sát gần realtime (kỳ vọng p50 E2E $<$ 100 ms khi đủ ba agent).
3. **H3 — Chịu lỗi:** Khi Analysis bị dừng, Decision vẫn phát alert `degraded` và Sensor/Decision không chết theo.
4. **H4 — Trade-off:** Có đánh đổi rõ giữa độ bền limping-mode và độ trễ E2E khi lỗi (bị chi phối bởi timeout).

6.2 Môi trường và phương pháp đo

Bảng 6.1: Cấu hình thí nghiệm (cửa sổ ổn định)

Tham số	Giá trị
Nền tảng	Docker Compose
Giới hạn mỗi agent	2 vCPU, 4 GB RAM, CPU-only
Cửa sổ đo	40 giây
Chu kỳ sensor	2 giây
Timeout Analysis	5 giây
Ngày đo (bản ghi chính)	2026-07-27
Nguồn số liệu	<code>reports/metrics_run.json</code>

E2E latency được định nghĩa là khoảng thời gian từ `ts` của `SensorReading` đến thời điểm Decision phát `DecisionAlert` cùng `trace_id`. Inference latency là `inference_ms`

do Analysis tự đo quanh lần gọi ONNX Runtime. Throughput suy từ số bản tin trong cửa sổ chia cho `duration_sec`. Tài nguyên lấy mẫu `docker stats` trong cùng cửa sổ.

6.3 Kết quả tài nguyên

Bảng 6.2: Mẫu `docker stats` cuối của sổ đo

Thành phần	CPU (mẫu)	RAM	Giới hạn
<code>sensor_agent</code>	0,08%	20,89 MiB	4 GiB
<code>analysis_agent</code>	0,21%	51,54 MiB	4 GiB
<code>decision_agent</code>	0,19%	20,11 MiB	4 GiB
<code>mqtt-broker</code>	0,54%	2,74 MiB	(host)

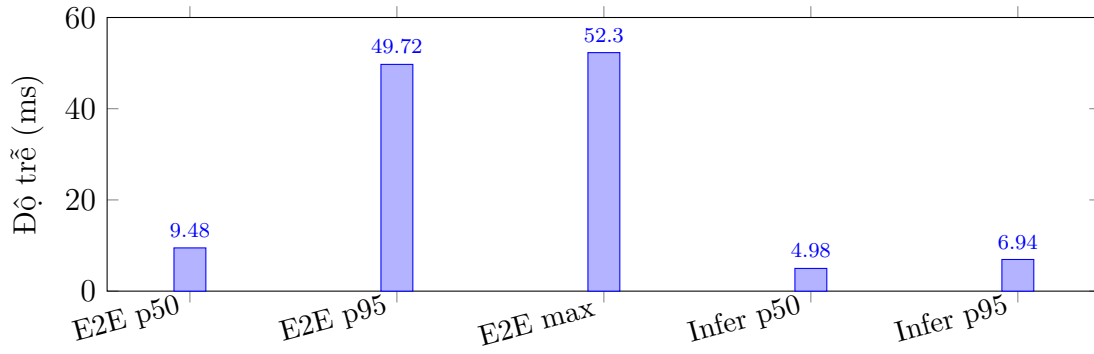
Nhận xét (H1). Toàn bộ agent dùng RAM $\ll$ 4 GB; Analysis cao nhất $\approx$ 51,5 MiB ($\approx$ 1,26% hạn mức). CPU mẫu cũng rất thấp ở chu kỳ cảm biến 2 s. Kết quả xác nhận ngân sách đề bài còn dư địa lớn với stack MQTT + ONNX hiện tại; phần dư địa này vừa là điểm mạnh (an toàn tài nguyên), vừa là cơ sở để thảo luận mở rộng model phức tạp hơn nếu đo lại dưới cùng limit.

6.4 Kết quả độ trễ và throughput

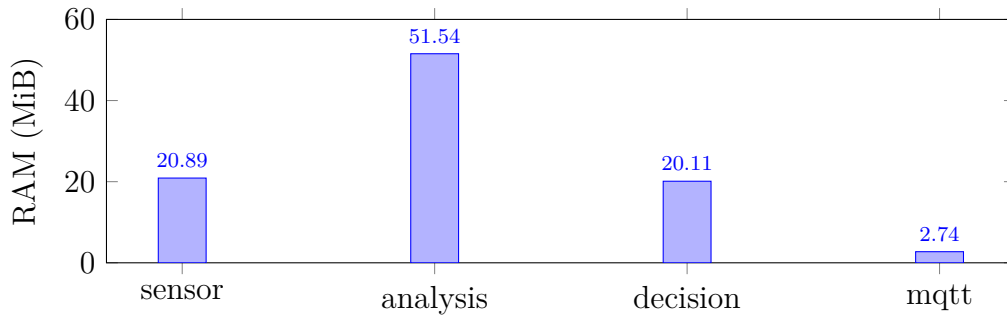
Bảng 6.3 tổng hợp toàn bộ chỉ số chính của cửa sổ ổn định (tránh tách nhiều bảng lặp cùng một bộ số). Hình 6.1 và Hình 6.2 trực quan hóa độ trễ và RAM.

Bảng 6.3: Tổng hợp kết quả thực nghiệm (Docker Compose, mỗi agent 2 vCPU / 4 GB, CPU-only; cửa sổ 40 s)

Nhóm chỉ số	Chỉ số	Giá trị
Tài nguyên	RAM <code>sensor_agent</code>	20,89 MiB
	RAM <code>analysis_agent</code>	51,54 MiB
	RAM <code>decision_agent</code>	20,11 MiB
	RAM <code>mqtt-broker</code>	2,74 MiB
Độ trễ	E2E latency p50	9,48 ms
	E2E latency p95	49,72 ms
	E2E latency trung bình / max	16,7 / 52,3 ms
	Inference ONNX p50	4,98 ms
	Inference ONNX p95	6,94 ms
Throughput	Readings / results / alerts (40 s)	21 / 21 / 21
	Thông lượng mỗi tầng	0,525 msg/s
	Alert <code>degraded</code> (ổn định)	0



Hình 6.1: So sánh phân vị độ trễ E2E và inference ONNX (cửa sổ 40 s)



Hình 6.2: RAM quan sát theo thành phần (mẫu `docker stats` cuối cửa sổ)

Nhận xét (H2). Số readings/results/alerts khớp 21/21/21 trong 40s cho thấy pipeline không mất tầng khi ổn định; throughput $\approx 0,525$ msg/s khớp chu kỳ sensor 2s. E2E p50 rất thấp ($\approx 9,5$ ms); inference chiếm phần đáng kể trong ngân sách thời gian đó (≈ 5 ms).

Phân tích đuôi p95. E2E p95 (49,72 ms) cao gấp khoảng 5 lần p50 (9,48 ms), trong khi inference p95 chỉ $\approx 6,9$ ms — gần với p50 inference. Điều này cho thấy đuôi E2E **không** chủ yếu đến từ ONNX, mà từ jitter ngoài suy luận: lập lịch container/OS, hàng đợi MQTT, và đồng bộ thời điểm đo theo `trace_id`. Max E2E (52,3 ms) sát p95, gợi ý đuôi chưa cực đoan trong mẫu $n = 21$, nhưng kích thước mẫu còn nhỏ nên khoảng tin cậy của p95 chưa chặt. Dù vậy, cả p95 lẫn max vẫn < 100 ms trong thí nghiệm — phù hợp giám sát môi trường không đòi hỏi điều khiển vòng kín cực nhanh.

6.5 Kết quả kịch bản lỗi

Nhận xét (H3, H4). Khi Analysis bị `docker stop`, Decision vẫn phát quyết định sau khoảng timeout cấu hình; Sensor tiếp tục publish. E2E khi degraded bị chi phối bởi 5s timeout (khoảng 5,0–5,2s), khác hoàn toàn chế độ bình thường mili-giây. Đây chính là trade-off độ bền limping-mode $\leftrightarrow$ độ trễ khi lỗi: có thể chỉnh `analysis_timeout_sec` theo SLA, nhưng timeout quá nhỏ sẽ tăng false-degraded khi Analysis chỉ chậm nhất thời.

Bảng 6.4: Fault path — dùng Analysis Agent

Quan sát	Kết quả
Hành vi Decision	Alert <code>degraded=true</code> sau $\sim$ timeout 5 s
E2E khi degraded (demo)	Khoảng 5010–5155 ms
Sensor / Decision	Tiếp tục sống và vận hành
Phục hồi	<code>docker compose start analysis_agent</code>

6.6 Phân tích trade-off

Bảng 6.5: Phân tích trade-off

Trục	Quan sát
Độ trễ $\leftrightarrow$ tài nguyên	E2E p50 $\sim$ 10 ms với RAM agent < 55 MiB; còn nhiều headroom trong 4 GB
Chất lượng $\leftrightarrow$ độ phức tạp	Isolation Forest đủ synthetic; chưa phản ánh nhiều/drift thực địa
Độ bền $\leftrightarrow$ độ trễ khi lỗi	Timeout 5 s giữ hệ sống nhưng kéo E2E khi degraded
Framework orchestration	Tự viết vòng MQTT tránh overhead framework nặng trên biên
Quan sát vận hành	Dashboard tốt cho demo; số liệu báo cáo lấy từ metrics file

Nhìn tổng thể, đồ án ưu tiên chứng minh ba trụ cột đề bài (tài nguyên, phối hợp/fault, đo lường) bằng một stack đủ nhỏ và đủ rõ, thay vì tối đa hoá độ phức tạp mô hình ngay từ đầu.

6.7 Đối chiếu với mục tiêu đề bài

Bảng 6.6: Đối chiếu mục tiêu đề bài

Mục tiêu	Mức đạt
≥ 3 agent vai trò khác nhau, 2c/4GB	Đạt
MQTT thật, không shared memory giả lập	Đạt
AI cục bộ trên edge	Đạt (ONNX)
Điều phối tập trung được mô tả và hiện thực	Đạt
≥ 1 fault scenario	Đạt
Metrics p50/p95, throughput, E2E	Đạt

6.8 Thảo luận hạn chế thực nghiệm

1. Cảm biến giả lập — chưa đánh giá drift, nhiễu cảm biến, hay thiếu mẫu thực địa; precision/recall trên dữ liệu thật chưa có.
2. Cửa sổ 40s đủ chứng minh pipeline và số liệu chính, nhưng chưa phải stress test dài hạn hay bão tải.
3. Chưa so sánh ONNX với LLM Q4 / hybrid cloud trên cùng bộ metric.
4. Chưa đo năng lượng, nhiệt độ thiết bị, hay triển khai trên phần cứng biên vật lý.
5. Mẫu `docker stats` là điểm cuối của sổ; phân tích CPU/RAM theo chuỗi thời gian dài hơn sẽ làm rõ spike (nếu có).

Các hạn chế trên không phủ nhận kết quả đã đạt theo tiêu chí đề, nhưng xác định biên độ suy luận hợp lệ của số liệu và định hướng Chương 7.

6.9 Tóm tắt chương

Thực nghiệm ủng hộ H1–H4 trong phạm vi thiết lập hiện tại: hệ gọn trong ngân sách biên, độ trễ mili-giây khi đủ ba agent, và chế độ degraded hoạt động đúng khi mất Analysis. Trade-off và hạn chế được nêu rõ để tránh kết luận vượt quá điều kiện thí nghiệm.

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

Đồ án đã thiết kế, triển khai và đánh giá định lượng một hệ **multi-agent trên edge** cho miền giám sát môi trường thông minh, bám các ràng buộc cốt lõi của học phần CE2206:

1. Ba agent tách biệt về vai trò và tiến trình, giới hạn **2 vCPU / 4 GB**, CPU-only.
2. Giao tiếp **MQTT** với schema thống nhất và **trace_id** phục vụ truy vết E2E.
3. Suy luận AI cục bộ bằng **Isolation Forest** xuất **ONNX**, inference p50 ≈ 5 ms.
4. Điều phối **tập trung nhẹ** tại Decision Agent, kèm fallback khi Analysis timeout.
5. Đánh giá định lượng: E2E p50/p95 $\approx 9,5/49,7$ ms; RAM đỉnh Analysis $\approx 51,5$ MiB.

Kết quả cho thấy, với lựa chọn giao thức và mô hình phù hợp, hệ multi-agent Edge AI có thể đồng thời hướng tới **độ trễ thấp, tiêu thụ tài nguyên nhỏ**, và **khả năng chịu lỗi limping-mode** — ba trụ cột đề bài nhấn mạnh — trong điều kiện thí nghiệm đã mô tả.

7.2 Hướng phát triển

1. **Dữ liệu thật**: nối cảm biến vật lý hoặc tập dữ liệu môi trường công khai; đánh giá lại precision/recall và hiệu chuẩn ngưỡng fallback.
2. **Điểm cộng đề bài**: so sánh ONNX với LLM nhỏ Q4 (0,5B–1,5B); đo RAM/latency theo mức quantize dưới cùng limit 4 GB.
3. **Hybrid edge–cloud**: đẩy summary/alert lên cloud khi có mạng; giữ quyết định tối thiểu trên biên khi mất mạng.
4. **Vận hành**: bổ sung xác thực MQTT, TLS, lưu trữ alert dài hạn và cảnh báo vận hành.

5. **Đo năng lượng:** ước lượng joule/inference trên thiết bị biên thật nếu có phần cứng.
6. **Mở rộng thí nghiệm:** cửa sổ dài hơn, thay đổi chu kỳ sensor/timeout, phân tích chuỗi thời gian CPU/RAM.

7.3 Bài học kinh nghiệm

- Đo metrics **từ sớm** giúp tránh tình trạng “chạy được nhưng không chứng minh được”.
- Giữ agent tách process ngay từ đầu tránh nợ kỹ thuật khi nộp theo tiêu chí VM/container riêng.
- Ưu tiên model nhỏ đủ đúng đề trước khi tối ưu điểm cộng bằng LLM; mọi mở rộng phải đo lại dưới đúng hạn mức.
- Fault path cần được thiết kế như một luồng nghiệp vụ (timeout + fallback + chống double-alert), không chỉ là “bắt exception rồi log”.

Tài liệu tham khảo

- [1] CE2206. Đề tài cuối kỳ: Triển khai hệ thống multi-agent trên edge, 2026. Tài liệu đề bài học phần, file de-tai-cuoi-ky-edge-ai.PDF.
- [2] Docker Inc. Docker compose resource constraints. <https://docs.docker.com/compose/compose-file/>, 2024.
- [3] Eclipse Foundation. Eclipse Mosquitto — an open source MQTT broker. <https://mosquitto.org/>, 2024.
- [4] Eclipse Foundation. Eclipse Paho MQTT python client. <https://www.eclipse.org/paho/>, 2024.
- [5] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the IEEE International Conference on Data Mining (ICDM)*, pages 413–422, 2008.
- [6] Microsoft. ONNX runtime documentation. <https://onnxruntime.ai/>, 2024.
- [7] OASIS. MQTT version 3.1.1. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>, 2014.
- [8] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [9] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.